VoteEase Voting System

COMPREHENSIVE TECHNICAL & ARCHITECTURAL DOCUMENTATION

Collegiate Single-Vote Integrity, Multi-Election Engine & Real-Time Auditing

Generated on: September 30, 2026 | System Version: 2.4.0 (Full-Stack)

1. Executive Overview

VoteEase is an enterprise-grade collegiate voting and election management system designed to enforce cryptographic ballot privacy, eliminate duplicate voter fraud, and provide administrators with verifiable real-time turnout metrics. The architecture is engineered around the principle of strict zero-trust voter validation combined with thread-safe atomic commit transactions.

- **Multi-Tenant Election Engine:** Allows institutions and election officers to manage multiple concurrent elections (e.g. Student Council, Department Representatives, Club Executives) with isolated voter rosters, candidate lineups, and unique passcodes.
- **Single-Vote Guarantee:** Guarantees that each eligible student roll number can cast only one verified ballot per election through atomic locking and state synchronization.
- **Zero-Trust Voter Flow:** Voters authenticate with a two-part credential: an Election Security Passcode followed by Roll Number verification.
- **Comprehensive Audit & Export:** Automates position tallies, margin-of-victory calculations, and multi-sheet Excel workbooks (.xlsx) alongside CSV raw audit logs.

2. System Architecture & Tech Stack

The platform is built as a unified TypeScript application leveraging React 19 for the interactive client interface and Express on Node.js for authoritative API processing, data storage, and Excel workbook compilation.

- **Frontend:** React 19, TypeScript, Tailwind CSS v4, Lucide React Icons, Motion animations, Chart.js.
- **Backend API Gateway:** Node.js, Express, tsx engine, esbuild bundling.
- **Persistence Engine:** Thread-safe synchronized database with atomic write locking (withLock) serialized to JSON and multi-tab Excel workbooks via ExcelJS.
- **Security Protocol:** Case-insensitive alphanumeric roll normalization, cryptographic vote reference generation, and sanitization of legacy browser state.

3. Core Domain Data Models

All domain records are defined with strict TypeScript interfaces across the frontend and backend, ensuring end-to-end type safety:

- **AdminAccount:** Attributes: id, fullName, collegeName, username, email, createdAt.
- **Election:** Attributes: id, adminId, electionName, collegeName, securityCode, positions (string[]), electionStatus ("Active" | "Closed"), createdAt, description, votersCount, votesCount, candidatesCount, turnoutPercentage, endDate.

- **User (Voter Record):** Attributes: rollNumber, studentName, status ("Active" | "Inactive"), hasVoted ("Yes" | "No"), votedAt (timestamp), electionId.
- **Candidate:** Attributes: candidateId, candidateName, position, group/affiliation, imagePath, status ("Active" | "Inactive"), electionId.
- **VoteRecord:** Attributes: voteId, rollNumber, candidateId, candidateName, position, date, time, electionId.

4. REST API Endpoints Specification

A. Admin Authentication & Accounts

- **POST /api/admin/register:** Creates an administrator profile. Newly created accounts start with zero default elections, voters, or votes.
- **POST /api/admin/login:** Authenticates admin credentials via username/email and password.

B. Multi-Election Management

- **GET /api/admin/elections?adminId=:id:** Fetches all elections belonging to the admin with calculated voter, candidate, and turnout counts.
- **POST /api/admin/elections:** Creates a new election with custom name, institution, security code, and ballot positions.
- **PUT /api/admin/elections/:id:** Updates election details, positions, or security passcode.
- **DELETE /api/admin/elections/:id:** Cascading deletion: removes election, candidate records, voters, and votes.
- **POST /api/admin/elections/:id/toggle-status:** Switches election status between "Active" and "Closed".

C. Voter Authentication & Voting

- **POST /api/voter/verify-code:** Validates election security code and returns election metadata.
- **POST /api/voter/validate-roll:** Checks voter roster eligibility (active status, not already voted).
- **GET /api/voter/ballot/:electionId:** Retrieves ballot choices grouped by position.
- **POST /api/voter/cast-vote:** Atomically logs ballot selections, records audit log, and sets hasVoted = Yes.

D. Analytics, Roster & Audit Export

- **GET /api/stats?electionId=:id:** Live turnout summary: total voters, voted count, remaining, turnout percentage.
- **GET /api/results?electionId=:id:** Aggregated vote tallies and declared winners per position.
- **POST /api/users/bulk:** Batch roster ingestion from pasted CSV/TSV data.
- **GET /api/export/:type?electionId=:id:** Generates and downloads certified Excel workbook (.xlsx) or CSV audit.

5. Security, Concurrency & Single-Vote Integrity

Online collegiate voting requires strict safeguards against vote duplication, race conditions, and ballot tampering:

- **Atomic Concurrency Locking:** The database implements a promise-chained execution queue (withLock) ensuring simultaneous vote submissions are queued sequentially, eliminating double-vote vulnerabilities.
- **Two-Factor Eligibility Gate:** A voter cannot reach the ballot without passing both the election-specific security passcode verification and an active, non-voted student roll check.
- **Zero-Default Account Architecture:** Newly registered administrators receive completely empty dashboards (0 voters, 0 active elections, 0 votes, 0 candidates) to guarantee absolute data privacy and isolation.
- **Cryptographic Receipts:** Upon casting a ballot, the student receives a unique printable receipt containing a Reference ID, timestamp, and position breakdown.

6. Deployment & Operational Runbook

The application is fully containerized and production-ready for standard Node.js hosting environments (Google Cloud Run, Docker, VPS, or Kubernetes).

- **Development Mode:** npm run dev — Runs Express server with Vite HMR middleware on port 3000.
- **Lint & Validation:** npm run lint — Executes TypeScript compiler (tsc --noEmit) for static verification.
- **Production Build:** npm run build — Compiles frontend client assets to /dist and bundles backend with esbuild into /dist/server.cjs.
- **Production Start:** npm run start — Executes the bundled Node.js server (node dist/server.cjs) serving both static assets and API routes.